Line-level Semantic Structure Learning for Code Vulnerability Detection

Ziliang Wang
Peking University
Beijing, China
wangziliang@pku.edu.cn

Ge Li*
Peking University
Beijing, China
lige@pku.edu.cn

Jia Li
Peking University
Beijing, China
lijia@stu.pku.edu.cn

Yihong Dong
Peking University
Beijing, China
dongyh@stu.pku.edu.cn

Yingfei Xiong
Peking University
Beijing, China
xiongyf@pku.edu.cn

Zhi Jin
Peking University
Beijing, China
zhijin@pku.edu.cn

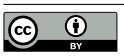
Abstract

Unlike the flow structure of natural languages, programming languages have an inherent rigidity in structure and grammar. However, existing detection methods based on pre-trained models typically treat code as a natural language sequence, ignoring its unique structural information. This hinders the models from understanding the code's semantic and structual information. To address this problem, we introduce the Code Structure-Aware Network through Line-level Semantic Learning (CSLS), which comprises four components: code preprocessing, global semantics awareness, line semantic awareness, and line semantic structure awareness. The preprocessing step transforms the code into two types of text: global code text and line-level code text. Unlike typical preprocessing methods, CSLS preserves structural elements such as line breaks and indentation characters while processing the global text. While preserving global code semantics, the CSLS network emphasizes capturing structural relationships between line semantics. By modeling each line's semantics, CSLS treats line-level semantics as the smallest structural unit to learn nonlinear structural relationships, thereby improving code vulnerability detection accuracy. We conducted extensive experiments on vulnerability detection datasets from real projects. Results show that our preprocessing method significantly enhances the performance of existing baseline models. Additionally, the CSLS model outperforms the state-of-the-art baselines in code vulnerability detection, achieving 70.57% accuracy on the Devign dataset and a 49.59% F1 score on the Reveal dataset. These results demonstrate the importance of preserving and utilizing code structure information to improve the performance of code vulnerability detection models.

Keywords

Vulnerability detection, Model collaboration, Large language model, Pre-trained models

*Corresponding author.



Internetware 2025, Trondheim, Norway

ACM ISBN 979-8-4007-1926-4/25/06
https://doi.org/10.1145/3755881.3755894

ACM Reference Format:
Ziliang Wang, Ge Li, Jia Li, Yihong Dong, Yingfei Xiong, and Zhi Jin. 2025. Line-level Semantic Structure Learning for Code Vulnerability Detection. In *the 16th International Conference on Internetware (Internetware 2025), June 20–22, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3755881.3755894

1 Introduction

Software code vulnerabilities refer to weaknesses in code that can be exploited, leading to severe consequences such as unauthorized information disclosure and cyber extortion [15, 38]. The growing magnitude of this issue is highlighted by recent statistics: in the first quarter of 2022, the US National Vulnerability Database (NVD) reported 8,051 vulnerabilities, a 25% increase from the previous year [8]. Additionally, a study found that 81% of 2,409 analyzed codebases contained at least one known open-source vulnerability [1]. The widespread nature and increasing number of these vulnerabilities underscore the urgent need for robust automated vulnerability detection mechanisms. Implementing such systems is crucial for enhancing software security and preventing a range of potential threats [15, 20, 21, 38].

Existing literature on vulnerability detection models can be broadly categorized into two main types: (1) traditional detection models [43, 44], (2) deep learning (DL)-based models [9, 12, 26, 35] Traditional detection models often require experts to manually develop detection rules [6, 14]. This approach is labor-intensive and struggle to maintaining low rates of false positives and false negatives [25, 26]. In contrast, deep learning (DL)-based detection methods learn vulnerability patterns from training datasets [9, 26, 27, 37], eliminating the need for manual heuristics and enabling autonomous feature identification. They avoid manual heuristic methods and autonomously learn and identify vulnerability features.

To further enhance the understanding of vulnerability semantics, recent studies have attempted to incorporate code structure into pre-trained code models. [11, 31]. Devign [47] method combines the Abstract Syntax Tree (AST), Control Flow Graph (CFG), Data Flow Graph (DFG), and code sequence and other structures to introduce structural information. In contrast to code block (multi-line code) based structure modeling, ReGvd [31] is a more elaborate way to track code structure by building a graph with tokens as the smallest unit. In the latest research, Trace [11] takes the execution path information into account in the pre-trained model, thus providing

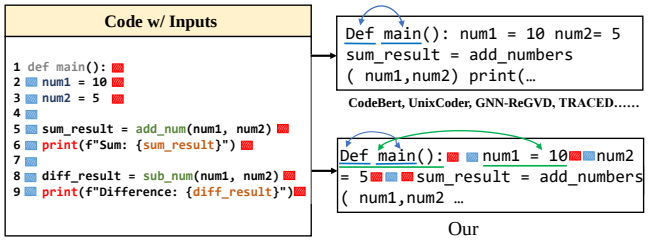


Figure 1: The existing preprocessing methods make the model only focus on the positional relationship between tokens and it is difficult to identify the structural relationship between code lines. CSLS makes it easier for the model to identify code lines and to learn relationships between line semantic .

an effective vulnerability detection model. However, these methods usually preprocess the code into natural language text and remove structural elements as shown in Figure 1 [13, 17][1] [37][2] [11][3]. The loss of these structural elements often leads models to focus solely on token relationships, neglecting the structure between lines of code. For example, a newline character can tell the model that the statement is over, and a blank line indicates that the code above and below it may have different functions, such as variable definition and function implementation. Even though these methods reintroduce code structure information in various ways. However, whether based on code blocks or tokens, it is still difficult to comprehensively construct the structural information of complex code texts with the help of a large number of complex modeling rules (such as CFG) [34, 41].

Our Approach. In this paper, we explore the extension of line-level code text to semantic space with the help of pre-trained models, and take line semantics as nodes to model the complex nonlinear relationship between code lines, so as to develop an effective vulnerability detection model. Thus, in the *Code Pre-processing Process*, we split the code into lines array. Then the *CSLS model* completes the line semantic structure modeling.

Code Pre-processing Process. To address the issues mentioned above, we proposed a new code preprocessing method to enhance code structure perception in vulnerability detection tasks. This process results in two different code texts: (1) *Global Code Text.* We replace the previous preprocessing with a simpler method to obtain global word segmentation, as shown in Figure 1. CSLS preserves line breaks and all whitespace before tokenization. (2) *Line-level Code Text.* By preserving code structure information, CSLS splits the code snippets into lines and generates a line-level token array for each snippet. This approach preserves the original code structure while enabling the model to learn from both global and line-level supervision.

CSLS Model. To further enhance code structure perception, we propose a new vulnerability detection network architecture using code models to learn code semantics and structure from different code texts. (1) *Global Semantic-Aware.* This model uses global word segmentation as input to perceive global and structural information. (2) *Line Semantic-Aware.* A pre-trained code model uses line-level code as input to capture the semantics of each line. (3) *Line Semantic Structure-Aware.* A Transformer module processes the semantics of each line, using line numbers as positional encoding. The multi-head attention mechanism learns the nonlinear structural relationships between line semantics. Finally, the CSLS model analyzes code vulnerability risks from three perspectives: global semantics, line semantics, and line semantic structure, achieving high-precision vulnerability detection.

Results. With our *Code Pre-processiong*, we report state-of-the-art results on popular or representative code models: CodeBERT's [13] accuracy increased from 63% to 65% on the Devign dataset[47], and UniXcoder-base's accuracy rose from 64.8%[31] to 68.8%. Other datasets, such as ReVeal[4], also show positive effects. Our proposed CSLS further enhances vulnerability detection accuracy. We achieved state-of-the-art results with CSLS, reporting 70.57% accuracy on Devign, 91.86% accuracy, and a 49.59% F1-score on ReVeal. In summary, the main contributions of this paper are:

a) We propose an enhanced code preprocessing method for vulnerability detection tasks, highlighting the flaws of existing preprocessing methods and emphasizing the importance of preserving code structure information.
b) We propose a vulnerability detection network architecture that enhances code structure awareness. The model uses the pre-trained model to learn the line-level semantics, uses the transformer to learn the nonlinear structural relationship of the line-level semantics, and combines it with the global information to achieve high-precision vulnerability detection.
c) We evaluate the proposed preprocessing method on two real datasets. The experimental results show that preserving code structure information effectively improves the performance of existing methods and provides new benchmark data for vulnerability detection.
d) We evaluate our network architecture on two real-world datasets. The results show that the proposed CSLS architecture further improves the accuracy of code vulnerability detection by perceiving code line-level semantic structures [4].

2 RATIONALE

In this section, we discuss the background of vulnerability detection using code structure information, explaining the efficiency and effectiveness of our approach. We also compare related works that incorporate code structure in model fine-tuning to highlight the novelty of our work.

2.1 Control Flow Graph (CFG) based on Code Block

A Control Flow Graph (CFG) illustrates the control flow within a program, where each node represents a basic block, and edges denote the control flow between these blocks. A basic block is a sequence of statements or instructions with a single entry and exit point, typically consisting of multiple lines of code executed sequentially. *(1) Basic Block Identification:* The first step is to identify

[1]https://github.com/microsoft/CodeXGLUE
[2]https://github.com/daiquocnguyen/GNN-ReGVD
[3]https://github.com/ARiSE-Lab/TRACED_ICSE_24.git
[4]Our anonymous replication package (data and code) :https://figshare.com/articles/dataset/CSLS_model_code_and_data/26391658

the basic blocks, which are determined by control flow instructions like conditional and unconditional jumps. A basic block usually consists of multiple lines of code executed sequentially between a single entry and exit point. *(2) Construction of Nodes and Edges:* Each basic block is represented as a node in the CFG, with edges added based on control flow instructions. *(3) Marking Entry and Exit Nodes:* Identify and mark the program's entry and exit nodes, with the entry node indicating the program's start and the exit node its end. Devign [47], a popular vulnerability detection method, uses the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Data Flow Graph (DFG) of code snippets as inputs, combining these representations for vulnerability detection.

2.2 Undirected Graphs based on Code Tokens

An existing fine-tuning method (ReGVD) [31] incorporates code structure into the process, primarily using graph neural networks (GNNs) to detect vulnerabilities in source code by representing the code structure as a graph. Graphs are constructed in two main ways: *(1) Unique Token-Focused Graph:* Nodes represent unique tokens in the source code. *(2) Index-Focused Graph:* Each position (index) in the token sequence of the source code is represented as a node, and edges are created between positions that co-occur within a fixed-size sliding window.

ReGVD uses a more flexible and general approach to building graphs based on token co-occurrence to capture semantic and structural relationships, differing from CFGs.

2.3 The Novelty of Our Work

These methods focus on the role of code structure in vulnerability detection from various perspectives [18, 31, 47]. However, heuristic approaches to structural modeling in low-dimensional spaces are limited. The structure of vulnerabilities caused by complex interactions between multiple lines of code is usually difficult to model by conventional structure modeling methods. Inspired by the state selection network [16], CSLS encodes lines of code into a high-dimensional space, modeling nonlinear structures in the semantic space with line semantics as the basic unit. Compared to existing structure-aware vulnerability detection methods, CSLS offers the following advantages: Preserving structural elements during global semantic modeling makes the model more effective in recognizing code context and distinguishing code blocks. In the line-structure-aware process, we model complex nonlinear relationships between lines of code in a high-dimensional semantic space. Our method captures the intricate structural relationships within the code, offering a more comprehensive and precise understanding of the code structure.

3 Approach

In this section, we introduce the Code Structure-aware Network through Line-level Semantic Learning (CSLS). Figure 2 provides an overview of CSLS, which comprises four main phases: (1) Data Preprocessing, (2) Line-level Semantic-aware, (3) Line-semantic Structure-aware, and (4) Global Semantic-aware.

3.1 Data Preprocessing

We denote the historical vulnerability dataset as $L = (P, Y)$, where p_i represents a code snippet, and y_i represents the corresponding binary label (i.e., vulnerable or secure) for the i-th snippet, with $0 < i \leq M$. Here, M is the total number of code snippets. The labels y_i can take values of either 0 or 1, where $y_i = 0$ indicates that the code is free of vulnerabilities, and $y_i = 1$ indicates the presence of a vulnerability in the code.

Global Preprocessing. Unlike existing methods that remove structural elements, we only truncate code fragments to fit the input size constraints of the model. This step is formalized as follows:

$$C_i = C_{i_0} C_{i_1} \dots C_{i_n}, \quad 0 < i \leq M \tag{1}$$

Here, C_{i_n} represents the n-th token of the i-th code fragment, with n being the maximum input length allowed by the code model.

Code Line Preprocessing. The number of lines in a code fragment typically exceeds the number of snippets. We define L_{i_n} as the n-th line of the i-th code fragment. To expedite the learning process for the code model, we implement the following preprocessing steps for each training batches:

Line Alignment: For all snippets in the same training batch, we align the number of lines in each snippet. Statistical analysis shows that the average number of lines of code in the Devign dataset training set is 62.21 under the 1024 length limit. Therefore, CSLS presets $MAX(k) = 100$ lines per code fragment split. For all code fragments n in the same batch, we use $k = \max(\text{len}(n.\text{line}), 100)$.

$$L_i = L_{i_0} L_{i_1} \dots L_{i_k}, \quad 0 < k \leq 100 \tag{2}$$

Here, L_{i_k} represents the k-th line of the i-th code fragment, with k being the maximum number of line.

Line-Token Alignment: Each line in the code snippet is tokenized individually. Tokens are then padded to ensure uniform length. For snippets within the same batch, we align the number of tokens per line. Statistical analysis shows that p=20 can cover 93.41% of the lines of code under the 1024 input limit. Thus, CSLS presets the maximum number of tokens per line to $p = 20$.

$$L_{i_k} = T_{i_{k0}} T_{i_{k1}} \dots T_{i_{kp}}, \quad p = 20 \tag{3}$$

Here, $L_{i_{kp}}$ represents the p-th token of the k-th line of the i-th code fragment, with p being the maximum number of tokens per line.

The preprocessed snippets have the same number of lines of code and number of tokens per line. This approach allows the model to generate semantics for all lines of the same training batch at the same time instead of learning one line at a time, which will greatly improve the training speed and reduce the learning cost.

3.2 Line-level semantic and structure aware

Line Semantic Learning. After the code preprocessing, we obtain an array of tokens based on line segmentation for all code fragments. And, in the same batch, this array has a consistent shape of $[b, k, p]$. Where b denotes the number of snippets in the batch, k is the number of lines in each snippet, and p is the maximum number of tokens per line.

We flatten the preprocessed token array to form a two-dimensional array z with size $[b * k, p]$. This allows us to process an entire

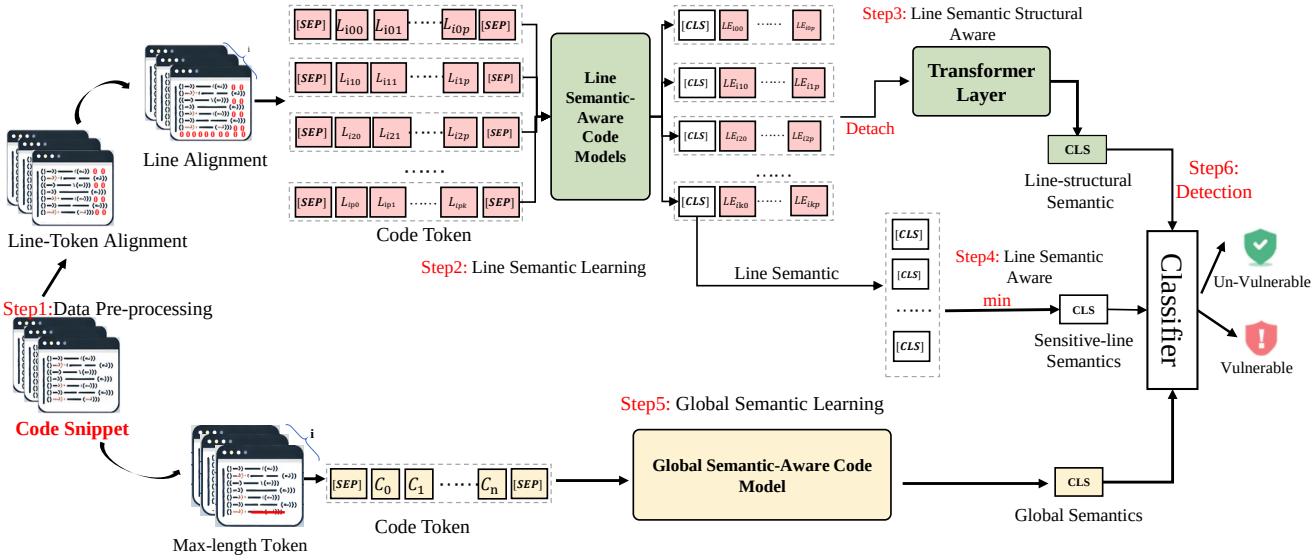


Figure 2: The CSLS framework implements vulnerability detection by capturing three different semantics: (1) Line-level Structural Semantics (Step 3), (2) Sensitive-Line Semantics (Step 4), and (3) Global Semantics (Step 5)..

batch of data in a single operation without iterating over each line. CSLS uses a line semantic-aware model, and this paper uses the UniXcoder-nine model by default. We process z to quickly obtain deep semantic representation through a single batch. The formalization process is as follows:

$$LE = f_l(z, z_{mask}), \tag{4}$$

where LE has the shape $[b \times k, h]$, with $h = 768$. Here, b represents the batch size, l represents the sequence length, and h represents the dimension of the hidden layer. The f_l stands for line semantic-aware model. z_{mask} is provided by the completion process during data pre-processing. The output LE of the line semantic-aware model contains multiple hidden states, and we use the hidden state of the first token of the last layer (usually the [CLS] token) to represent the semantics of each lines.

Finally, to restore the processed data to its original batch structure, we reshape LE to $[b, k, h]$, where each element now contains an embedded representation of the corresponding line of code. This refactoring facilitates subsequent steps such as further analysis or specific line-level tasks.

Line semantic Structure Aware. After the above process, CSLS obtains the semantic array LE for each line of the code fragment. In this module, CSLS uses a Transformer model to model line semantics to learn nonlinear structural information between lines.

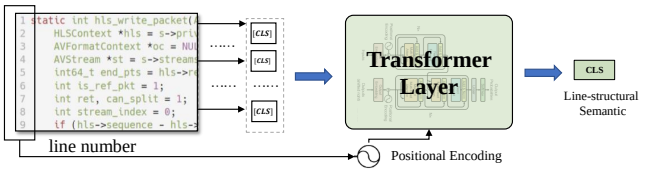


Figure 3: A Transformer model is used to capture the semantic structural relationships between lines.

As shown in Figure 3, the input line semantics LE preserves the order of line numbers, and the default positional encoding of the Transformer layer corresponds to these line numbers.

This alignment allows the Transformer to capture the structural relationships between lines of code. The output of the Transformer layer is then used to assess the vulnerability of the code based on its structural representation. The formalization is as follows:

$$S_{repr} = f_t(D(LE_1..LE_k)) \tag{5}$$

Where f_t represents a standard Transformer model with 8 layers and 8 attention heads.

The multi-head attention mechanism within the Transformer enhances this process by allowing each attention head to focus on different aspects of the relationships between line semantics, enabling a more comprehensive and nonlinear modeling of the line-level semantic structure.

Semantic Detach. In this step, we introduce a crucial step $D(LE)$ by detaching the sentence embeddings from the global encoder before passing them to the Transformer encoder.

This detachment prevents gradients from flowing back into the global encoder during the optimization of the Transformer encoder.

The purpose of this separation is to ensure that the Transformer encoder focuses exclusively on learning the structural relationships between code lines, rather than reprocessing semantic information already captured by the line-semantic model.

This delineation of responsibilities allows the line-semantic model to specialize in semantic understanding and the Transformer encoder to specialize in structural dependency modeling.

Sensitive-line Semantic Awareness. In the line semantic-aware module, we obtain the overall semantic representation of each line of code by calculating the average of the 768-dimensional vectors in the semantic representation of each line. For the code snippets in each batch, we identify the most representative lines of

code by computing the minimum of the semantic representation of each line. The specific process is as follows:

For the semantic representation matrix LE for each sentence, we first compute the average of each token over 768 dimensions:

$$LE_{mean} = \frac{1}{h}\sum_{j=1}^{h} LE_{i,j} \tag{6}$$

Then, we find the row index with the smallest mean for each batch:

$$L_{min} = Index(\arg\min(LE_{mean})) \tag{7}$$

Finally, we select the semantic representation of the row with the smallest mean value from each batch to form the final sensitive-line semantic representation:

$$L_{repr} = LE[L_{min}] \tag{8}$$

where L_{repr} represents the semantic representation of a code fragment whose line semantics is closest to 0. The reason why we update the row semantics in reverse in this way is that label 0 is the label of the vulnerability, so a code segment is vulnerable if the line of code with the highest risk (closest to 0) is vulnerable. We cannot use the each line semantics and fragments labels for loss calculation, because not every line is vulnerable, even for vulnerable code fragments.

3.3 Global Semantic Awareness

Global Semantic Learning. In the global semantic learning module, we use a code model to capture the global semantic information of code fragments. We feed the code tokens C_i into the global semantic-aware model (UniXcoder-nine):

$$G_{repr} = f_g(C_i) \tag{9}$$

where G_{repr} is the first token CLS of the model output, which usually represents the global semantics.

Vulnerability Prediction. In the vulnerability prediction module, we concatenate the global semantic G_{repr}, the line-sensitive semantic L_{repr}, and the line semantic structure representation S_{repr} to form the final representation vector H:

$$H = [S_{repr}, L_{repr}, G_{repr}] \tag{10}$$

Then, we input the final representation vector into a Multi-Layer Perceptron (MLP) classifier for vulnerability prediction:

$$\hat{y} = \sigma(f_m(H)) \tag{11}$$

Where σ is the Sigmoid activation function, and P is the predicted vulnerability probability. The f_m is a classification network.

3.4 Loss Function

The loss function adopted for the code models training is the cross-entropy loss [47], commonly used in classification problems for its effectiveness in penalizing the predicted labels and the actual labels:

$$H(y, \hat{y}) = -y\log(\hat{y}) - (1-y)\log(1-\hat{y}) \tag{12}$$

where y is the actual label, $\hat{y}$ is the predicted value.

4 Study Design

4.1 Datesets

To assess the effectiveness of CSLS, we utilize two datasets from real-world projects: (1) Devign [47] and (2) Reveal [4].

Label balanced. The Devign dataset, sourced from a graph-based code vulnerability detection study [47], comprises function-level C/C++ source code from the well-known open-source projects QEMU and FFmpeg. Following the methodology outlined by Li et al. [47], the dataset is divided into training, validation, and testing sets using a standard 80:10:10 split. Security researchers labeled the vulnerable code through a meticulous two-stage review process. The ratio of positive and negative samples in this dataset is close to 1:1, which is label balanced.

Label unbalanced. The REVEAL dataset, as detailed in [4], addresses issues of data redundancy and unbalanced class distributions in existing datasets, making it suitable for software vulnerability detection tasks. This dataset includes source code from the Linux Debian kernel and Chromium projects and features an imbalanced label distribution, with a 10:1 ratio of normal to vulnerable code fragments. The REVEAL dataset also follows an 80:10:10 split for training, validation, and testing. Throughout the experiments, the proportion of positive and negative samples in the training, validation, and testing sets remained consistent with the original dataset distributions.

4.2 Performance Metrics

In the process of evaluating the performance of the model, the proposed method employs four metrics[47]:

Precision: This metric is defined as the quotient of true positives (TP) and the sum of true positives and false positives (FP), providing a measure of the accuracy of instances identified as positive. Formally, it is given by: $Precision = \frac{TP}{TP+FP}$.

Recall: Recall measures the fraction of actual positive instances that are correctly identified. It is calculated as the ratio of true positives to the sum of true positives and false negatives (FN): $Recall = \frac{TP}{TP+FN}$.

F1 Score: The F1 is the harmonic mean of precision and recall, providing a single metric that balances both concerns: $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$.

Accuracy: Accuracy reflects the proportion of true positive and true negative instances among all evaluated instances, offering an overall measure of the model's performance: $Accuracy = \frac{TP+TN}{TP+TN+FN+FP}$.

4.3 Baseline Methods

In our evaluation, we compare CSLS with nine state-of-the-art methods.

(1) ChatGPT [33]: The ChatGPT model demonstrates the capabilities of deep learning in code generation and processing, though it is not specifically designed for software vulnerability detection. We tested the dateset on chain-of-though based cue words in ChatGPT 3.5 and ChatGPT 4o, repeated the process three times, and averaged the results [40]. We provide the template for the prompts we use in the code bundle and supplementary materials.

Table 1: Comparison results for different models on Devign and Reveal datasets.

	Devign [47]				Reveal [4]			
Models	**Acc**	**Recall**	**Prec**	**F1**	**Acc**	**Recall**	**Prec**	**F1**
ChatGPT 3.5 COT	49.83	32.24	33.00	30.61	63.72	26.34	30.54	27.70
ChatGPT 4o COT	53.73	7.46	45.94	4.06	20.97	22.17	**97.33**	12.51
Devign	56.89	52.50	64.67	57.59	87.49	*31.55*	36.65	33.91
ReGVD	61.89	48.20	60.74	53.75	*90.63*	14.47	*64.70*	23.65
CodeBERT	63.59	41.99	66.37	51.43	90.41	25.87	54.62	35.11
UniXcoder-base	65.77	51.55	*66.42*	*58.05*	90.50	**39.91**	53.52	*45.72*
CodeT5+	*65.62*	*55.29*	64.73	*59.64*	***90.94***	31.14	59.16	40.80
UniXcoder-nine	***66.98***	56.33	**66.63**	***61.05***	90.72	33.77	56.20	42.19
TRACED	64.42	**61.27**	60.03	**61.05**	**91.11**	21.49	**68.05**	32.66
DeepDFA	45.61	**98.72**	45.61	**62.65**	53.64	**53.15**	49.10	***51.04***
PDBERT	***67.16***	51.92	***68.92***	59.24	***91.20***	30.26	**62.72**	40.82
CSLS	**70.57**	***59.36***	**71.70**	**64.95**	**91.86**	*39.91*	***65.46***	**49.59**

(2) Devign [47]: Devign utilizes a graph-based model with a Gated Graph Recurrent Network (GGN) to represent the graph that combines the Abstract Syntax Tree (AST), Control Flow Graph (CFG), Data Flow Graph (DFG), and code sequence of the input code fragment for vulnerability detection.

(3) ReGVD [31]: By transforming the source code into a graph structure, ReGVD uses the label embedding in GraphCodeBERT to learn the code structure and enhance the vulnerability detection ability of the code model.

(4) CodeBERT [13]: CodeBERT is a pre-trained model that integrates natural language and programming language representations, supporting a wide range of coding tasks, including code understanding and generation.

(5) CodeT5+ [39]: CodeT5+ is an encoder-decoder model for code, with flexible component modules that can be combined to handle a variety of downstream code tasks.

(6) UniXcoder [17]: UniXcoder extends the capabilities of models like CodeBERT by incorporating a deep understanding of code syntax and semantics, thereby enhancing model performance on tasks such as code summarization, translation, and completion. UniXcoder-nine, its latest extension in 2023, further trains UniXcoder-base to develop robust code models.

(7) TRACED [11]: TRACED introduces an execution-aware pre-training strategy for source code by integrating execution traces with source code and executable inputs. This method enhances the model's ability in tasks like static execution estimation, clone retrieval, and vulnerability detection.

(8) DeepDFA [37]: DeepDFA employs dataflow analysis-inspired graph learning to detect vulnerabilities efficiently, outperforming non-transformer baselines and retaining high accuracy with minimal training time.

(9) PDBERT [29]: PDBERT integrates novel pre-training objectives for predicting statement-level control and token-level data dependencies, thereby enabling fine-grained semantic analysis and achieving state-of-the-art results on multiple vulnerability detection and assessment tasks.

5 EXPERIMENTS

We aim to answer the following Research Questions (RQs):

RQ1: How effective is CSLS compared with the state-of-the-art baselines on vulnerability detection?

RQ2: What are the effects of the code pre-processing process on vulnerability detection methods based on fine-tuning of pre-trained models?

RQ3: What is the difference in the performance of the model using different pre-trained codes?

5.1 Implementation

CSLS contains a pre-trained model fine-tuning process and is therefore affected by the learning rate and Batch size. We provide detailed runtime environment and script setup in the open source code repository. Due to page limitations, we do not discuss the impact of different Settings in this category. In general, a batchsize of 12 and a learning rate of 2e-5 is a recommended choice.

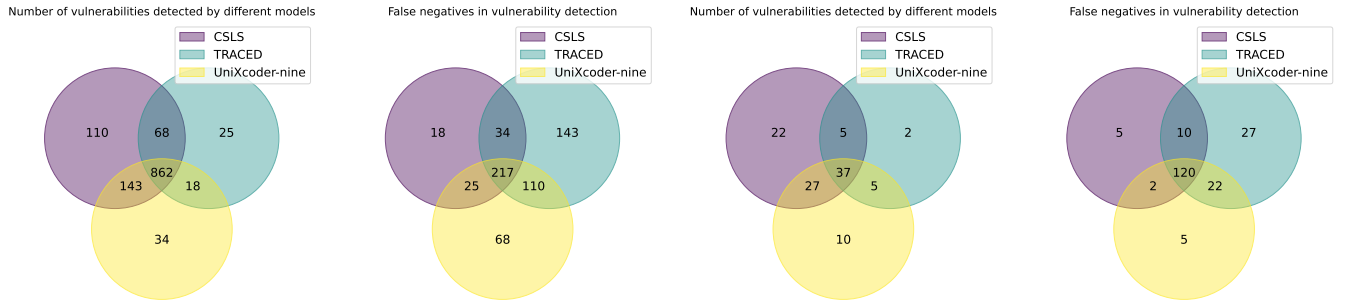
All experimental procedures are saved with the random seed used in the existing literature: seeds=123456 [31]. All experiments are repeated 3 times to ensure repeatability.

Scope: CSLS currently identifies vulnerabilities at the function level. However, recent works, such as that by Li et al. [23], utilize deep learning explanation tools to report vulnerabilities at the line level. By leveraging our approach's ability to model line-level semantics, we can directly pinpoint indices of line-level vulnerabilities, identifying them as dangerous rows. Nevertheless, comprehensive verification of these indices necessitates substantial expert knowledge or a standardized verification methodology, which we defer to future work.

5.2 RQ1. Effectiveness of CSLS

To answer the first question, we compare CSLS with the seven baseline methods on the two datasets as shown in Table 1. We can draw conclusions about the performance of CSLS compared to the baselines across the evaluated datasets.

Table 1 presents the performance of ChatGPT on the vulnerability detection task. It is evident that large-scale language models employ aggressive detection logic. Specifically, in ChatGPT 4o, nearly all code snippets were identified as vulnerable, leading to considerably low F1 scores across both datasets.



(a) Number of vulnerabilities discovered in dataset Devign

(b) Number of missed reports in dataset Devign

(c) Number of vulnerabilities discovered in dataset Reveal

(d) Number of missed reports in dataset Reveal

Figure 4: Comparison of vulnerability detection performance of different models on two datasets.

Table 2: Number of tokens for different datasets after preprocessing. T stands for the number of tokens.

Data	Our(T)	Exist(T)	Ratio	line_num	T>1024
Devign	723.70	559.74	77%	113.37	14.64%
Reveal	488.40	457.34	94%	32.07	8.95%
PrimeVul*	449.12	360.73	80%	44.94	0%

Table 3: Comparison results for different models on Devign dataset. (S) stands for preserving structural information. L represents the maximum input length of the model.

	Devign [47]				
Models	L	Acc	Recall	Prec	F1
CodeBERT	400	63.59	41.99	66.37	51.43
CodeBERT(S)	400	65.95	44.46	70.54	54.54
UniXcoder-base	1024	65.77	51.55	66.42	58.05
UniXcoder-base(S)	1024	68.85	58.64	68.91	63.36
CodeT5+-base	512	65.62	55.29	64.73	59.64
CodeT5+-base(S)	512	66.43	44.38	71.77	59.64
TRACED	512	64.42	61.27	60.03	61.05
TRACED(S)	512	67.45	54.26	68.37	60.50
UniXcoder-nine	1024	66.98	56.33	66.63	61.05
UniXcoder-nine(S)	1024	69.10	56.73	70.28	62.78
CSLS	1024	70.57	59.36	71.70	64.95

Table 4: Results of statistical tests for model comparison.

Models compared	χ^2 statistic	p-value
CodeBERT vs. CodeBERT(S)	697.50	1.04×10^{-153}
Traced vs. Traced(S)	799.29	7.66×10^{-176}
UniXcoder-b vs. UniXcoder-b(S)	809.34	5.01×10^{-178}
UniXcoder-n vs. UniXcoder-n(S)	696.84	1.45×10^{-173}
CSLS vs. UniXcoder-n(S)	1433.51	0

CSLS demonstrates a marked superiority on both datasets. In the Devign dataset, CSLS attains the highest Accuracy of 70.57%, the highest F1 score of 64.95% and the highest Precision outperforming all other models. This improvement can be attributed to the retention of structural information during pre-processing, which enhances the model's understanding of code semantics. DeepDFA employing a small-parameter model, effectively learned from limited training data and achieved rapid convergence, yet its overall accuracy and F1 on Devign remained relatively modest; meanwhile, PDBERT leveraged control-dependency and data-dependency predictions to significantly deepen its code semantic understanding, thereby delivering higher F1 and precision on Devign. We provide a Venn diagram of the performance of different models for vulnerability detection in Fig. 4, where the false positive rate of each method does not significantly differ according to the metric data; therefore, we pay more attention to the accuracy and false negative rate of the model for vulnerabilities. As shown in Fig. 4(a), CSLS detected 110 different vulnerabilities, while TRACED detected 68 and UniXcoder-nine detected 25. This indicates that CSLS has a higher detection capability, identifying more vulnerabilities than the other models. In terms of missed reports, CSLS had the fewest false negatives (18), whereas TRACED and UniXcoder nine missed 34 and 143 vulnerabilities, respectively. This demonstrates CSLS's superior accuracy in minimizing missed detection as shown in Fig. 4(b). This indicates that CSLS has the most balanced performance in correctly identifying vulnerabilities without being skewed towards over-predicting (which would increase recall but decrease precision) or under-predicting (which would do the opposite).

On the Reveal dataset, since the proportion of negative samples in this dataset is 90, the model with strong fitting performance generally exceeds 90% on ACC. In this dataset, people are generally interested in the ability of the model to find positive samples (vulnerability). For CSLS, both Recall and F1 metrics maintain the level of optimal level. As show in Fig. 4(c), CSLS detected 22 vulnerabilities, outperforming TRACED (7) and UniXcoder-nine (2). This again shows CSLS's higher effectiveness in detecting vulnerabilities. In Fig. 4(b), CSLS had 10 missed reports, which is significantly lower than TRACED (27) and UniXcoder nine (5), indicating a balanced performance in both detecting vulnerabilities and minimizing false negatives. These figures not only show that CSLS maintains its high performance in different testing conditions but also that it consistently understands and predicts code vulnerabilities with high precision and recall.

Table 5: Comparison results for different models on Reveal dataset. (S) stands for preserving structural information.

	Reveal [4]			
Models	**Acc**	**Recall**	**Prec**	**F1**
CodeBERT	90.10	28.50	51.18	36.61
CodeBERT(S)	89.18	29.80	44.15	35.60
UniXcoder-base	90.50	39.91	53.52	45.72
UniXcoder-base(S)	90.50	42.98	53.26	47.57
CodeT5+-base	90.94	31.14	59.16	40.80
CodeT5+-base(S)	91.20	29.38	63.20	40.11
TRACED	90.12	24.14	67.07	35.48
TRACED(S)	91.95	24.12	84.61	37.54
UniXcoder-nine	90.14	31.57	51.42	39.13
UniXcoder-nine(S)	91.33	27.63	66.31	39.00
CSLS	91.86	39.91	65.46	49.59

5.3 RQ2: Effect of different code pre-processing procedures on detection performance

To illustrate the impact of the code pre-processing process on the vulnerability detection task, we conduct experiments on several models. Since we propose to keep line characters and Spaces in code snippets, this increases the number of tokens per sample. This can lead to bad results in cases where the input length of the pre-trained model is limited. Therefore, we analyze the token length after code-word segmentation.

As can be seen from Table 2, the situation of the number of tokens after PBE word segmentation after different preprocessing methods have been adopted for different data sets. The data shows that the Devign dataset is severely affected, with the number of tokens reduced by 23% after processing by the currently commonly used pre-processing method (Exist). At the same time, the Devign dataset is also far more than other datasets in the number of lines of code. The code snippet in Reveal is much less affected. There is possible reasons for this. First, the complexity of the code function of the projects in Reveal is lower than that of Devign, and the average number of tokens is only half of that of Devign. At the same time, the proportion of code functions with more than 1024 tokens in Devign is more than 14.64, which is much higher than that of Reveal data. In order to better verify the effectiveness, we sampled a new vulnerability detection benchmark PrimeVul [10], which has more diverse vulnerability types and more extreme data imbalance. There are some samples in this dataset that are far longer than the input length of the baseline model, so we remove the samples that tokens are beyond 1024 to obtain PrimeVul*.

Table 3 shows the vulnerability detection performance on the dataset Devign under different data pre-processing processes. The experimental results show that all the models have a great degree of performance improvement compared with the indicators reported in the literature. For example, CodeBERT, whose best known performance reported in the literature was only 63%@ACC, went up to 65.95%@ACC using the new code pre-processing pipeline without making any model modifications. At the same time, it is clear that models with input sequence lengths of 1024 get a bigger boost because they can see more structural information while preserving the semantics of the code. Models with lower input lengths often have to trim their code. Table 4 presents the statistical results of vulnerability detection based on different preprocessing methods. The data show that different preprocessing methods make significant differences in model checking results.

Table 5 shows the situation on Reveal, where there is a significant difference between Reveal and the dataset Devign. Firstly, the two processing methods have less impact on this dataset than Devign. Secondly, there are far more non-vulnerability data than vulnerability data in this data, which increases the difficulty of vulnerability detection. The performance of CodeBERT decreases after our way of pre-processing. This is due to the limited input, the model learns fewer tokens after introducing structural information, and the unbalanced samples lead to a decrease in the modeling ability of the model. As the input length of the model increases, the other models get better, and the performance of different models on this dataset improves. This may be due to the fact that this dataset has much less structural information than Devign (only 6%). As shown in Table 6, in the PrimeVul dataset, the trend is consistent as other datasets.

Table 6: Comparison results for different models on PrimeVul dataset. (S) stands for preserving structural information.

	PrimeVul [10]			
Models	Acc	Recall	Prec	F1
Unixcoder-BASE	55.76	67.17	55.12	60.54
Unixcoder-BASE (S)	56.68	70.21	55.66	62.09
Unixcoder-nine	56.22	49.54	57.80	53.35
Unixcoder-nine (S)	56.83	68.96	55.91	61.76
Our (Unixcoder-nine)	56.98	76.59	55.38	64.28

Table 7: Effects of different code models. CSLS(X) means that the global semantic model and the row-level semantic model use the model.

	Devign [47]			
Models	Acc	Recall	Prec	F1
CodeBERT(CB)	63.59	41.99	66.37	51.43
CSLS(CB)	65.84	44.39	70.32	54.42
TRACED(T)	64.42	61.27	60.03	61.05
CSLS(T)	67.60	57.37	67.28	61.93
UniXcoder-base(Un-b)	65.77	51.55	66.42	58.05
CSLS(Un-b)	69.43	58.08	70.23	63.58
UniXcoder-nine(Un-n)	66.98	56.33	66.63	61.05
CSLS(Un-n)	70.57	59.36	71.70	64.95

5.4 RQ3. Effects of using different code models in CSLS

In this section, we investigate the effects of using different code models in the Code-Semantic Learning System (CSLS). The experiment involves two code models, allowing for various combinations. The results, as summarized in Table V, demonstrate that our method provides enhancements regardless of the base model used.

Table 8: Comparison results for different Hyperparameter.

Hyper-parameter		Devign [47]			
P	K	Acc	Recall	Prec	F1
20	70	69.32	**63.10**	67.86	65.40
20	100	**70.57**	59.36	71.70	64.95
20	120	69.21	51.47	**73.57**	61.07
10	70	69.61	61.59	68.95	**65.06**
10	100	69.83	60.23	69.93	64.72
10	120	69.10	59.92	68.80	64.05

Table 7 presents a comparison of different models on the Devign dataset. The metrics considered include Accuracy (Acc), Recall, Precision (Prec), and F1 score. Here are the detailed observations from the results: Table 7 shows that combining different code models leads to improved performance, with UniXcoder models showing significant gains. The CSLS(Un-N) achieves the highest metrics, including an accuracy of 70.57%, recall of 59.36%, precision of 71.70%, and an F1 score of 64.95%. These results indicate that leveraging different model configurations can significantly enhance vulnerability detection performance.

From these results, it is evident that using different combinations of code models in CSLS leads to consistent improvements across all evaluated metrics. Notably, the combinations involving UniXcoder exhibit significant enhancements, with the CSLS(Un-N) standing out as the most effective. These experimental results verify that our method can be effectively extended based on different code models for different application scenarios.

5.5 Hyperparameter Experiments

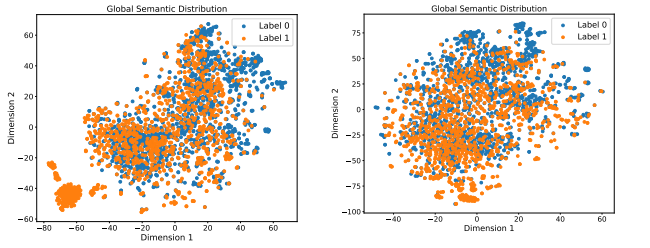
CSLS has two hyperparameters: the number of lines k and the default number of tokens per line p. Given the strong domain-specific nature of the vulnerability detection task, it is advisable to choose the settings of these hyperparameters based on the scenario. For the experiments in this paper, we selected the hyperparameter settings according to the code style of the projects included in the Devign dataset.

As shown in Table 8, we experimented with different combinations of p and k. Specifically, we tested p values of 10 and 20, and k values of 70, 100, and 120. The results indicate that the best performance in terms of accuracy (70.57) and F1-score (64.95) was achieved with $p = 20$ and $k = 100$. We did not explore other combinations of p and k as the average number of lines in the code snippets of this dataset is 117, with an average of 15 tokens per line.

6 Discussion

6.1 Effectiveness Analysis of Structural Information

To provide a more qualitative perspective on how structural information affects CSLS in practice, we conducted a focused experiment examining how the model distinguishes vulnerable (positive) and non-vulnerable (negative) samples when code structure is preserved. Specifically, we compared two preprocessing schemes on the Devign dataset: (i) a scheme that retains line breaks, indentation, and other structural cues, and (ii) a scheme that normalizes



(a) Pre-processing that preserves the code structure

(b) Pre-processing that does not preserve the code structure

Figure 5: Comparison of vulnerability detection performance of different prep-rocessing process.

the code (e.g., removes line breaks and extra spaces) so that only token sequences remain. We fine-tuned a UniXcoder-based variant of CSLS for both preprocessing schemes and then projected the global semantic representations (i.e., the [CLS] embeddings) of the resulting models onto a 2D plane via t-SNE. Figures 5(a) and 5(b) illustrate these projections, where label "0" denotes vulnerable code and label "1" denotes non-vulnerable code.

In Figure 5(a), which retains structural information, vulnerable samples (blue points) appear more tightly grouped, suggesting the model readily captures vulnerability patterns that are associated with distinctive line-level properties (e.g., poorly validated input checks, risky API calls, or suspicious pointer manipulations localized to specific lines). In contrast, when structural cues are removed in Figure 5(b), we observe more intermixing of blue and orange points, implying that the model struggles to separate vulnerable from non-vulnerable snippets based solely on token sequences.

For non-vulnerable samples (orange points), Figure 5(a) shows a relatively compact cluster in the lower-left quadrant, indicating that many secure code snippets share similar structural features (e.g., consistent indentation and short, clearly separated function blocks). When the structure is preserved, CSLS can more easily discern that code arranged in methodical, standardized patterns tends to be safe. By contrast, in Figure 5(b), non-vulnerable snippets disperse more broadly, implying that code structure (e.g., line boundaries, indentation depth) was helpful in reinforcing their "secure" traits for the model.

In essence, structural cues help CSLS group common "safe" patterns and highlight code lines that deviate from these patterns, enabling it to more effectively flag potential flaws. While we have not shown detailed per-line "case studies" (which may require extensive expert input), these visual mappings and observations illustrate how structural information specifically strengthens the positive/negative distinction. Hence, this qualitative analysis highlights the mechanism behind CSLS's improved performance and shows how preserving line-level code structure can yield clearer separation between vulnerable and secure functions.

7 THREATS TO VALIDITY

Threats: We used multiple code models as well as a transformer model for the vulnerability detection task, which may increase the model parameters (297.74M). This can lead to higher training and deployment costs. However, due to the scarcity of vulnerability

data, researchers generally can only use smaller code models (110M-770M) to complete fine-tuning, so we believe that this overhead is in an acceptable range. Inside. At the same time, the batch design with line-level semantic awareness does not show a multiple increase in memory footprint.

Internal Validity: During validation on different datasets, we found that CSLS is affected by code style. Code with a minimalist style is likely to be less affected by the approach as they almost have similar code structure. The vulnerability detection of code with complex structure is more beneficial than the structure-based detection model. At the same time, even if we maintain a good expectation for the input length of the pre-trained code model, it is undeniable that our pre-training method increases the input length, which may lead to the performance of CSLS being affected in some restricted scenarios.

8 Related work

8.1 Traditional Vulnerability Detection

Over the years, numerous methods for vulnerability detection have been developed. Initially, research in this area predominantly focused on identifying vulnerabilities through manually customized rules [6, 14]. Static analysis tools rely on manual rules and precise specification of code behavior, which are difficult to obtain automatically. While these heuristic approaches offered solutions for vulnerability detection, they necessitated extensive manual analysis and the formulation of defect patterns.

8.2 Deep Neural Network for Vulnerability Detection

To perceive code text nonlinear characteristics, recent research has turned to the model based on neural network, hole features extracted from code snippets [9, 35]. Existing deep learning-based vulnerability detection models are predominantly divided into two categories: token-based and graph-based models.

Token-based models treat code as a linear sequence and use neural networks (e.g., LSTM or Transformer) to learn vulnerability features from known cases, aiming to identify vulnerability features [7, 26, 35]. Concurrently, Li et al.[26] employed BiLSTM [36] to encode a segmented version of input code, known as 'code gadgets,' centered on key markers, especially library/API function calls. However, these token-based models often overlook the complexity of the source code structure, potentially leading to inaccurate detection.

In parallel, another research direction explores the potential of graph-based methods for vulnerability detection [4, 23, 31, 46]. For example, DeepWukong [7] uses GNNs for feature learning, focusing on compressing code fragments into a dense, low-dimensional vector space to enhance the detection of various vulnerability types. Graph-based detection models learn code structure through various graph representations, utilizing neural networks for vulnerability detection [3, 42]. For instance, Zhou et al.[47] used a gated graph recurrent network[24] to extract structural details from triadic graph representations—AST, CFG, and DFG. Chakraborty et al.[4] introduced REVEAL, an innovative approach that combines a gated graph neural network, re-sampling techniques[5], and triplet loss [30].

8.3 Pre-Trained Models for Vulnerability Detection

Inspired by the success of pre-trained models in natural language processing (NLP), recent research has increasingly focused on leveraging these models to enhance code vulnerability detection accuracy [2, 13, 17, 22, 28, 32].

The core concept behind these works is to use a model pre-trained on a large corpus of source code data, followed by specialized fine-tuning for specific tasks [22]. For instance, Feng et al.[13] proposed CodeBERT, designed specifically for understanding and generating source code, combining the processing capabilities of both natural and programming languages. Similarly, CuBERT employs masked language modeling with sentence prediction for code representation[22]. Additionally, some pre-trained models incorporate structural information of code fragments during the initial training phase [28, 32]. For example, Guo et al.'s GraphCodeBERT [18] leverages graph structures to infer data flow in code fragments. The objective is specifically tailored to address the structural dimension of programming languages. In comparative evaluations, CodeBERT is positioned as a baseline standard for various code-related tasks, including code clone detection and code translation. UniXcoder, a unified cross-modal pre-trained programming language model, also serves as a baseline method, trained on extensive code and natural language data [17].

Hanif et al. proposed VulBERTa, which pretrains a RoBERTa model with a custom tokenization pipeline for real-world C/C++ projects [19]. Nguyen et al. introduced ReGVD, combining graph structure and pre-trained models to address source code vulnerability detection [31]. Zhang et al. decomposed the code segment Control Flow Graph (CFG) into multiple execution paths and used the pre-trained model CodeBERT for vulnerability detection [45]. and Thapa et al. [38] explored the performance of fine-tuned language models for multi-class classification of similar types of vulnerabilities.

9 CONCLUSION

In this paper, we introduced the Code Structure-Aware Network through Line-level Semantic Learning (CSLS) to enhance code vulnerability detection. Our approach involves a refined code text processing workflow that retains structural elements, such as newlines and indentation, before modeling. This allows our proposed CSLS architecture to effectively capture and utilize line-level structural and semantic information. The CSLS architecture integrates four components: code preprocessing, global semantics-aware, line semantic-aware, and line semantic structure-aware. The core idea of CSLS architecture is to realize nonlinear structure modeling of row-level semantics in the row-level semantic space to achieve high-precision vulnerability detection.

Future work will focus on further refining the model architecture and exploring its applicability to other programming languages and broader categories of software vulnerabilities. Additionally, we aim to investigate the impact of different pre-processing techniques and model configurations to further enhance the detection capabilities of CSLS.

Acknowledgments

This research is supported by the National Key R&D Program under Grant No. 2023YFB4503801, the National Natural Science Foundation of China under Grant No. 62192733, 62192730, and the Major Program (JD) of Hubei Province (No.2023BAA024).

References

[1] 2022. Open source security and risk analysis report. *Available: https://www.synopsys.com/content/dam/synopsys/ sig-assets/reports/rep-ossra-2022.pdf* (2022).

[2] Jiangang Bai, Yujing Wang, Yiren Chen, Yaming Yang, Jing Bai, Jing Yu, and Yunhai Tong. 2021. Syntax-BERT: Improving Pre-trained Transformers with Syntax Trees. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 3011–3020.

[3] Sicong Cao, Xiaobing Sun, Lili Bo, Rongxin Wu, Bin Li, and Chuanqi Tao. 2022. MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks. In *Proceedings of the 44th International Conference on Software Engineering*. 1456–1468.

[4] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2021. Deep learning based vulnerability detection: Are we there yet. *IEEE Transactions on Software Engineering* (2021).

[5] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research* 16 (2002), 321–357.

[6] Checkmarx. 2022. Online. *Available: https://www.checkmarx.com/* (2022).

[7] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 3 (2021), 1–33.

[8] Xiao Cheng, Guanqin Zhang, Haoyu Wang, and Yulei Sui. 2022. Path-sensitive code embedding via contrastive learning for software vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 519–531.

[9] Hoa Khanh Dam, Truyen Tran, Trang Pham, Shien Wee Ng, John Grundy, and Aditya Ghose. 2017. Automatic feature learning for vulnerability prediction. *arXiv preprint arXiv:1708.02368* (2017).

[10] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624* (2024).

[11] Yangruibo Ding, Benjamin Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. 2024. Traced: Execution-aware pre-training for source code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.

[12] Xu Duan, Jingzheng Wu, Shouling Ji, Zhiqing Rui, Tianyue Luo, Mutian Yang, and Yanjun Wu. 2019. VulSniper: Focus Your Attention to Shoot Fine-Grained Vulnerabilities.. In *IJCAI*. 4665–4671.

[13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547.

[14] Flawfinder. 2022. Online. *Available: http://www.dwheeler.com/ flawfinde/r* (2022).

[15] Michael Fu and Chakkrit Tantithamthavorn. 2022. Linevul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 608–620.

[16] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).

[17] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225.

[18] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, LIU Shujie, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*.

[19] Hazim Hanif and Sergio Maffeis. 2022. Vulberta: Simplified source code pre-training for vulnerability detection. In *2022 International joint conference on neural networks (IJCNN)*. IEEE, 1–8.

[20] Julian Jang-Jaccard and Surya Nepal. 2014. A survey of emerging threats in cybersecurity. *Journal of computer and system sciences* 80, 5 (2014), 973–993.

[21] Arnold Johnson, Kelley Dempsey, Ron Ross, Sarbari Gupta, Dennis Bailey, et al. 2011. Guide for security-focused configuration management of information systems. *NIST special publication* 800, 128 (2011), 16–16.

[22] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and evaluating contextual embedding of source code. In *International conference on machine learning*. PMLR, 5110–5121.

[23] Yi Li, Shaohua Wang, and Tien N Nguyen. 2021. Vulnerability detection with fine-grained interpretations. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 292–303.

[24] Yujia Li, Richard Zemel, Marc Brockschmidt, and Daniel Tarlow. 2016. Gated Graph Sequence Neural Networks. In *Proceedings of ICLR'16*.

[25] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2244–2258.

[26] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018).

[27] Guanjun Lin, Jun Zhang, Wei Luo, Lei Pan, and Yang Xiang. 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2539–2541.

[28] Jinfeng Lin, Yalin Liu, Qingkai Zeng, Meng Jiang, and Jane Cleland-Huang. 2021. Traceability transformed: Generating more accurate links with pre-trained bert models. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 324–335.

[29] Zhongxin Liu, Zhijie Tang, Junwei Zhang, Xin Xia, and Xiaohu Yang. 2024. Pre-training by predicting program dependencies for vulnerability analysis tasks. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[30] Chengzhi Mao, Ziyuan Zhong, Junfeng Yang, Carl Vondrick, and Baishakhi Ray. 2019. Metric learning for adversarial robustness. *Advances in neural information processing systems* 32 (2019).

[31] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, and Dinh Phung. 2022. ReGVD: Revisiting graph neural networks for vulnerability detection. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 178–182.

[32] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguo Huang, and Bin Luo. 2022. Spt-code: Sequence-to-sequence pre-training for learning source code representations. In *Proceedings of the 44th International Conference on Software Engineering*. 2006–2018.

[33] openAI. 2022. Online. *Available: https://www.chat.openai.com/* (2022).

[34] Anh Viet Phan, Minh Le Nguyen, and Lam Thu Bui. 2017. Convolutional neural networks over control flow graphs for software defect prediction. In *2017 IEEE 29th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 45–52.

[35] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 757–762.

[36] Mike Schuster and Kuldip K Paliwal. 1997. Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing* 45, 11 (1997), 2673–2681.

[37] Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[38] Chandra Thapa, Seung Ick Jang, Muhammad Ejaz Ahmed, Seyit Camtepe, Josef Pieprzyk, and Surya Nepal. 2022. Transformer-based language models for software vulnerability detection. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 481–496.

[39] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).

[40] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[41] Ruoting Wu, Yuxin Zhang, Qibiao Peng, Liang Chen, and Zibin Zheng. 2022. A survey of deep learning models for structural code understanding. *arXiv preprint arXiv:2205.01293* (2022).

[42] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. 2022. VulCNN: An image-inspired scalable vulnerability detection system. In *Proceedings of the 44th International Conference on Software Engineering*. 2365–2376.

[43] Fa bian Yamagu chi. 2017. Pattern-based methods for vulnerability discovery. *it-Information Technology* 59, 2 (2017), 101–106.

[44] Fabian Yamaguchi. 2015. *Pattern-Based Vulnerability Discovery*. Ph. D. Dissertation. University of Göttingen.

[45] Junwei Zhang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2023. Vulnerability Detection by Learning from Syntax-Based Execution Paths of Code. *IEEE Transactions on Software Engineering* (2023).

[46] Weining Zheng, Yuan Jiang, and Xiaohong Su. 2021. Vu1SPG: Vulnerability detection based on slice property graph representation learning. In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE,

457–467.
[47] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems* 32 (2019).